

Breadth First Traversal (BFS) on a 2D array

Given a matrix of size **M x N** consisting of integers, the task is to print the matrix elements using [Breadth-First Search](#) traversal.

Examples:

Input: `grid[][] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}}`

Output: `1 2 5 3 6 9 4 7 10 13 8 11 14 12 15 16`

Input: `grid[][] = {{-1, 0, 0, 1}, {-1, -1, -2, -1}, {-1, -1, -1, -1}, {0, 0, 0, 0}}`

Output: `-1 0 -1 0 -1 -1 1 -2 -1 0 -1 -1 0 -1 0 0`

Approach: Follow the steps below to solve the problem:

1. Initialize the direction [vectors](#) `dRow[] = {-1, 0, 1, 0}` and `dCol[] = {0, 1, 0, -1}` and a [queue](#) of [pairs](#) to store the indices of matrix cells.
2. Start [BFS traversal](#) from the first cell, i.e. `(0, 0)`, and [enqueue](#) the index of this cell into the [queue](#).
3. Initialize a boolean [array](#) to mark the visited cells of the matrix. Mark the cell `(0, 0)` as visited.
4. Declare a function `isValid()` to check if the cell coordinates are valid or not, i.e. lies within the boundaries of the given Matrix and are unvisited or not.
5. Iterate while the [queue is not empty](#) and perform the following operations:
 - [Dequeue](#) the cell present at the [front of the queue](#) and print it.
 - Move to its adjacent cells that are not visited.
 - Mark them visited and enqueue them into the queue.

Note: Direction vectors are used to traverse the adjacent cells of a given cell in a given order. For example `(x, y)` is a cell whose adjacent cells `(x - 1, y)`, `(x, y + 1)`, `(x + 1, y)`, `(x, y - 1)` need to be traversed, then it can be done using the direction vectors `(-1, 0)`, `(0, 1)`, `(1, 0)`, `(0, -1)` in the up, left, down and right order.

Below is the implementation of the above approach:

// C++ program for the above approach

```
#include <bits/stdc++.h>
using namespace std;
```

```
#define ROW 4
#define COL 4
```

```
// Direction vectors
int dRow[] = { -1, 0, 1, 0 };
int dCol[] = { 0, 1, 0, -1 };
```

```

// Function to check if a cell
// is be visited or not
bool isValid(bool vis[][COL],
             int row, int col)
{
    // If cell lies out of bounds
    if (row < 0 || col < 0
        || row >= ROW || col >= COL)
        return false;

    // If cell is already visited
    if (vis[row][col])
        return false;

    // Otherwise
    return true;
}

// Function to perform the BFS traversal
void BFS(int grid[][COL], bool vis[][COL],
         int row, int col)
{
    // Stores indices of the matrix cells
    queue<pair<int, int> > q;

    // Mark the starting cell as visited
    // and push it into the queue
    q.push({ row, col });
    vis[row][col] = true;

    // Iterate while the queue
    // is not empty
    while (!q.empty()) {

        pair<int, int> cell = q.front();
        int x = cell.first;
        int y = cell.second;

        cout << grid[x][y] << " ";
    }
}

```

```

        q.pop();

        // Go to the adjacent cells
        for (int i = 0; i < 4; i++) {

            int adjx = x + dRow[i];
            int adjy = y + dCol[i];

            if (isValid(vis, adjx, adjy)) {
                q.push({ adjx, adjy });
                vis[adjx][adjy] = true;
            }
        }
    }
}

// Driver Code
int main()
{
    // Given input matrix
    int grid[ROW][COL] = { { 1, 2, 3, 4 },
                           { 5, 6, 7, 8 },
                           { 9, 10, 11, 12 },
                           { 13, 14, 15, 16 } };

    // Declare the visited array
    bool vis[ROW][COL];
    memset(vis, false, sizeof vis);

    BFS(grid, vis, 0, 0);

    return 0;
}

```

Output:

1 2 5 3 6 9 4 7 10 13 8 11 14 12 15 16

Time Complexity: $O(N * M)$

Auxiliary Space: $O(N * M)$

Depth First Traversal (DFS) on a 2D array

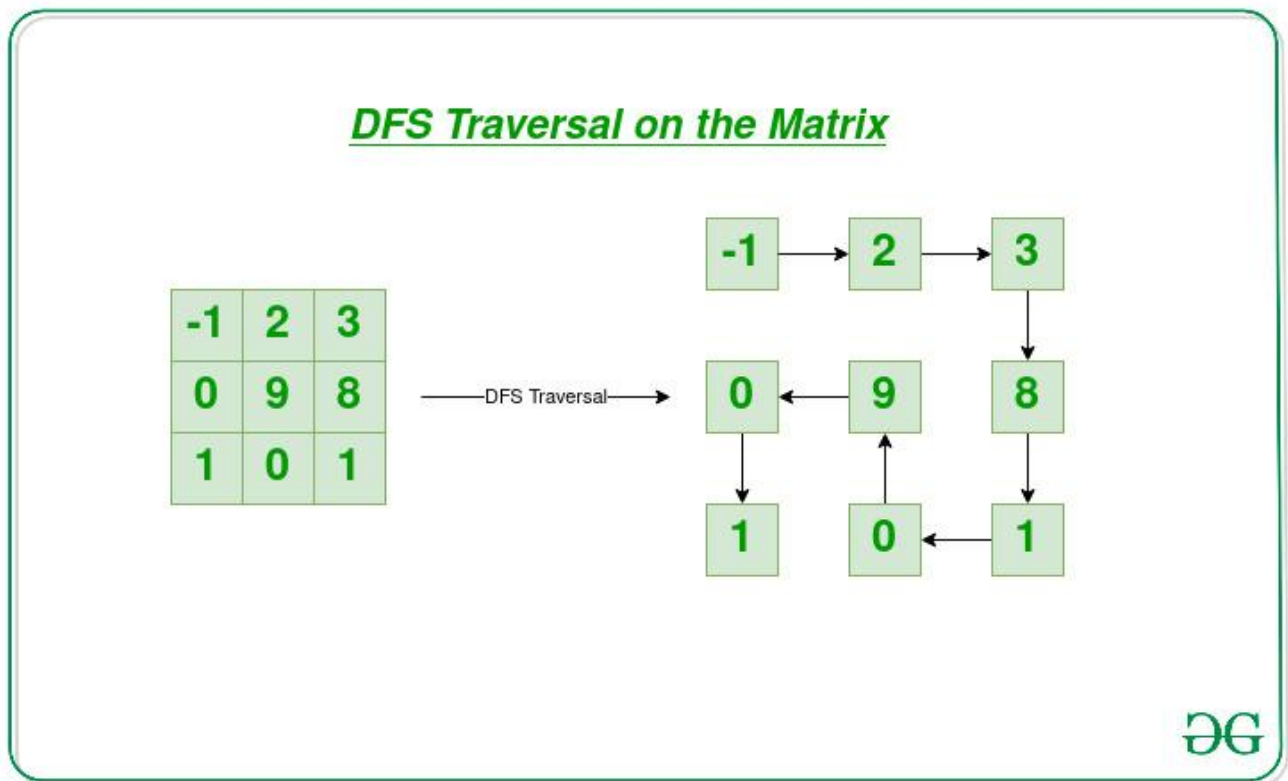
Given a 2D array `grid[ ][ ]` of dimension $N * M$, the task is to perform the Depth - First Search traversal on the given 2D array.

Examples:

Input: `grid[ ][ ] = {{-1, 2, 3}, {0, 9, 8}, {1, 0, 1}}`

Output: -1 2 3 8 1 0 9 0 1

Explanation: The sequence of traversal of matrix elements using DFS is -
1, 2, 3, 8, 1, 0, 9, 0, 1.



Input: `grid[ ][ ] = {{1, 2, 3}, {5, 6, 7}, {9, 10, 11}}`

Output: 1 2 3 7 11 10 6 5 9

Approach: The idea is to use [Stack Data Structure](#) to perform **DFS Traversal** on the [2D array](#). Follow the steps below to solve the given problem:

- Initialize a [stack](#), say **S**, with the starting cell coordinates as **(0, 0)**.
- Initialize an auxiliary boolean [2D array](#) of dimension **N * M** with all values as **false**, which is used to mark the visited cells.
- Declare a function **isValid()** to check if the cell coordinates are valid or not, i.e. lies within the boundaries of the given matrix and is unvisited or not.
- Iterate while the [stack is not empty](#) and perform the following steps:
 - **Pop** the cell present at the [top of the stack](#) and print the element at that cell.
 - **Push** the cell adjacent to the above-popped cells into the stack, if they are valid by checking using **isValid()** function.

Note: Direction vectors are used to traverse the adjacent cells of a given cell in a given order. For example, **(x, y)** is a cell whose adjacent cells **(x - 1, y)**, **(x, y + 1)**, **(x + 1, y)**, **(x, y - 1)** need to be traversed, then it can be done using the direction vectors **(-1, 0)**, **(0, 1)**, **(1, 0)**, **(0, -1)** in the up, left, down and right order.

Below is the implementation of the above approach:

C++

```
// C++ program of the above approach
#include <bits/stdc++.h>
using namespace std;
#define ROW 3
#define COL 3

// Initialize direction vectors
int dRow[] = { 0, 1, 0, -1 };
int dCol[] = { -1, 0, 1, 0 };

// Function to check if mat[row][col]
// is unvisited and lies within the
// boundary of the given matrix
bool isValid(bool vis[][COL],
            int row, int col)
{
    // If cell is out of bounds
    if (row < 0 || col < 0
        || row >= ROW || col >= COL)
        return false;

    // If the cell is already visited
    if (vis[row][col])
        return false;

    // Otherwise, it can be visited
```

```

        return true;
    }

// Function to perform DFS
// Traversal on the matrix grid[]
void DFS(int row, int col,
        int grid[][COL],
        bool vis[][COL])
{
    // Initialize a stack of pairs and
    // push the starting cell into it
    stack<pair<int, int> > st;
    st.push({ row, col });

    // Iterate until the
    // stack is not empty
    while (!st.empty()) {
        // Pop the top pair
        pair<int, int> curr = st.top();
        st.pop();
        int row = curr.first;
        int col = curr.second;

        // Check if the current popped
        // cell is a valid cell or not
        if (!isValid(vis, row, col))
            continue;

        // Mark the current
        // cell as visited
        vis[row][col] = true;

        // Print the element at
        // the current top cell
        cout << grid[row][col] << " ";

        // Push all the adjacent cells
        for (int i = 0; i < 4; i++) {
            int adjx = row + dRow[i];
            int adjy = col + dCol[i];
            st.push({ adjx, adjy });
        }
    }
}

// Driver Code
int main()
{
    int grid[ROW][COL] = { { -1, 2, 3 },
                           { 0, 9, 8 },
                           { 1, 0, 1 } };

    // Stores whether the current
    // cell is visited or not
    bool vis[ROW][COL];
    memset(vis, false, sizeof vis);

```

```
    // Function call
    DFS(0, 0, grid, vis);

    return 0;
}
```

Output:

-1 2 3 8 1 0 9 0 1

Time Complexity: $O(N * M)$

Auxiliary Space: $O(N * M)$